Agentes basados en objetivo

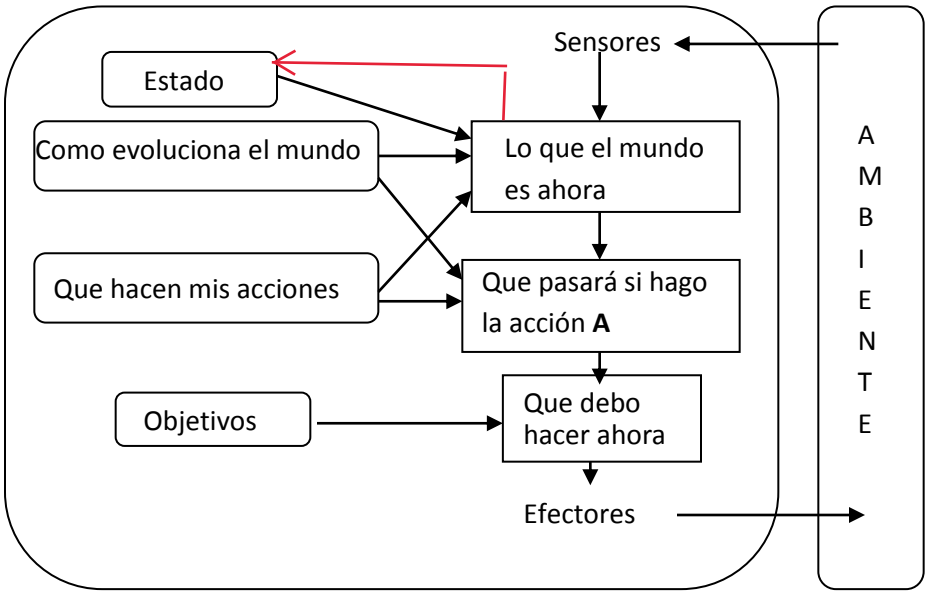
En este caso no es suficiente saber acerca del estado actual del ambiente para tomar la decisión de que hacer. Ejem: en un cruce, el taxi no sabría si a la derecha, a la izquierda, o derecho.El objetivo es entonces cierta información que describe situaciones que son deseables (destino del pasajero) y con ello seleccionar las acciones para lograrlo.

A veces es necesario considerar una larga secuencia de acciones para encontrar un camino que lleve al objetivo, y entonces se requiere la utilización de técnicas de IA (búsqueda, planeación, optimización, etc.).

A diferencia del agente reflexivo, donde el diseñador pre-calcula la acción correcta para varios casos, en el agente basado en objetivos se debe de "razonar". Por ejemplo el agente frena cuando ve luces rojas (es el caso de agente reflexivo).

- Razona si el carro de enfrente, tiene luces "encendidas" o se esta parando lentamente. El objetivo aquí, sería no golpear a otros carros por la acción de frenar.
- Si empieza a llover, debe considerar el estado de los frenos, velocidad, visibilidad, etc. Para poder tomar acciones. Esto quiere decir que el comportamiento relevante será alterado en función de las nuevas condiciones.
- En un agente reflexivo simple las reglas trabajarán para un solo destino; y deben reemplazarse cuándo se quiere algo nuevo. Es por eso que no están explícitamente en el caso de la estructura del agente basado en objetivos

Estructura



Agente basado en utilidad

Aquí se hace referencia a la utilidad como la calidad (o atributo) de ser útil.

Para llegar a un destino puede haber varios caminos y en consecuencia, la secuencia de acciones durante el trayecto es diferente. Con ello podríamos tener:

- Más rapidez
- Más seguridad
- Menor costo
- Más confiabilidad

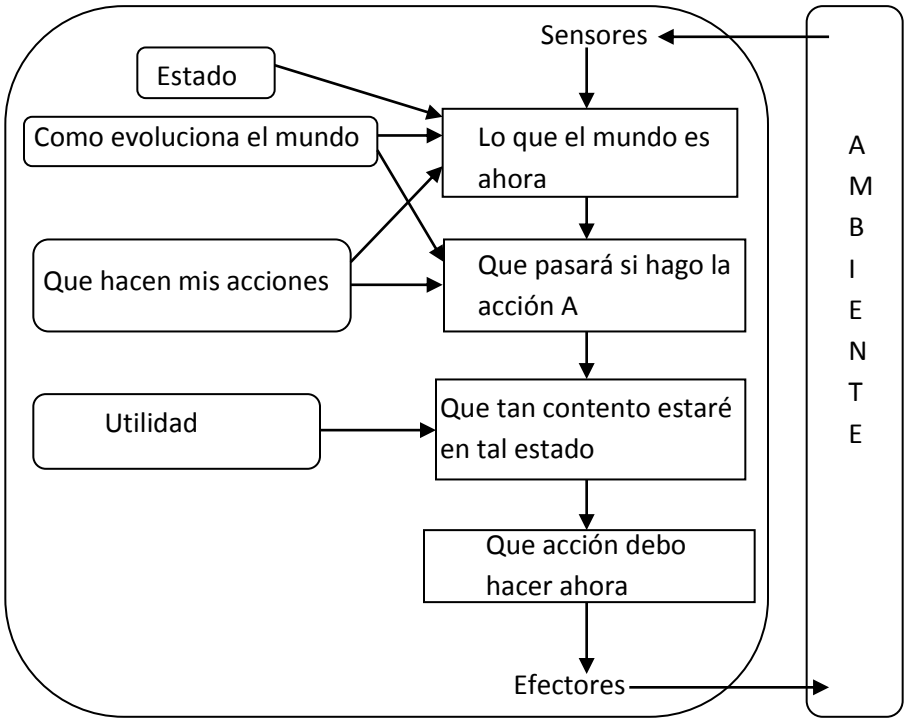
Aquí si un estado del mundo es preferible a otro, entonces dicho estado tiene mayor utilidad para el agente.

Utilidad : s → **R**

R describe el grado de "felicidad"

Esta utilidad permite tomar decisiones en caso de conflicto:

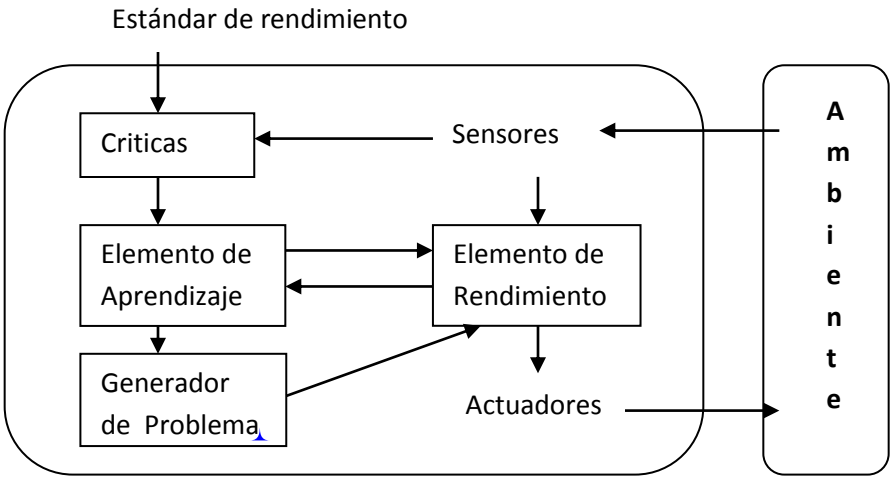
- Cuando dos objetivos se consideran y solo uno puede lograrse (velocidad, seguridad)
- Cuando hay varios objetivos que el agente tiene en la mira pero ninguno puede alcanzarse con certidumbre. La utilidad aquí provee un modo en el cual la verosimilitud del éxito se sopesa frente a la importancia de los objetivos.



Técnicamente, en este sentido el agente elije la acción que maximiza la **utilidad esperada** de los resultados de la acción.

Agentes de aprendizaje

Los modelos anteriores sobre agentes pueden mejorar su rendimiento a través del aprendizaje.



Críticas: Le dice al elemento de aprendizaje como está funcionando el agente respecto de un estándar de rendimiento.

Elemento de Aprendizaje: Responsable de hacer mejoras.

Elemento de Rendimiento: Toma percepciones y decide sobre que acciones realizar.

Generador de Problema: Sugiere acciones que guíen hacia nuevas experiencias e información al respecto (acciones exploratorias).

En el ejemplo del Taxi,

-El elemento del rendimiento sería cualquier colección de conocimientos y procedimientos que el taxi tiene para seleccionar sus acciones de manejo.

-El crítico observa al mundo y pasa información al elemento de aprendizaje. Si gira repentinamente y atraviesa 3 líneas, el crítico oye las protestas y eso lo pasa dicho módulo, y éste debe formular una nueva regla que modificaría al elemento de rendimiento.

- El generador de problemas puede sugerir experimentos tales como frenar eventualmente en diferentes caminos bajo distintas condiciones.

Formalizando

$S = \{s_1, s_2, \dots\}$ conjunto de estados del ambiente del agente

$A = \{a_1, a_2, \dots\}$ La capacidad de acción de un agente

En sentido abstracto un agente puede ser modelado en forma estándar como una función,

$acción: S^* \rightarrow A$ Aquí se hace referencia a la historia o experiencias a la fecha.

Un agente reflexivo carece de esta propiedad.

La experiencia (que el agente ha encontrado) es una secuencia de estados del ambiente.

Por otra parte la función,

$$amb : S \times A \rightarrow \varphi(S)$$

Nos modela el comportamiento no determinístico de un ambiente e indica que, se toma un estado actual s y una acción a tal que $s \in S$ y $a \in A$, para "mapearse" a un conjunto de estados $amb(s, a)$ ~~que pueden~~ resultar de la acción realizada a en s.

Si los conjuntos en el rango de amb son relacionados unívocamente, entonces el ambiente es determinístico y su comportamiento puede predecirse con precisión.

La interacción de agente y ambiente (historia) se puede representar:

Si $acción : S^* \rightarrow A$ es un agente, y

$amb : S \times A \rightarrow \varphi(S)$ es un ambiente, y

s_0 es el estado inicial del ambiente, entonces la secuencia

$$h : s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots\dots \xrightarrow{a_{u-1}} s_u \xrightarrow{a_u} \dots$$

representa una posible historia del agente en el ambiente.

La estructura de un agente esta basada en un mapa de lo interno de el:

- Estructura de datos
- Las operaciones sobre esas estructuras
- Control de flujo entre esas estructuras

Hay funciones de decisión para la percepción y la acción del agente.

Función *ver*: captura la habilidad del agente para observar su ambiente, y la *salida* de esta función *ver* sería una *percepción*

Por ejemplo: cámaras de video, sensores infrarrojo, robot móvil, etc. (todos ellos hardware cuando el agente está en un mundo físico).

En agentes de software los sensores serían los mandos que tratan de obtener información acerca del ambiente de software (ls, finger, etc).

Si P es un conjunto no vacío de percepciones, entonces la función

$$ver : S \rightarrow P$$

mapearía estados del ambiente a percepciones, y la *acción* es ahora una función

$$acción : P^* \rightarrow A$$

La cual mapea secuencias de percepciones a acciones

A veces estados de ambiente diferentes son "mapeados" a la misma percepción, o sea el agente recibiría la misma información perceptual de estados de ambiente diferentes, llamándolos indistinguibles. O sea tenemos

$s_1 \in S$ y $s_2 \in S$, tal que $s_1 \neq s_2$ pero la función

$ver(s_1) = ver(s_2)$

Ejem: termostato en una habitación, y representadas las siguientes sentencias con *x* y *y*

x = la temperatura es ok

y = Juan Pérez es primer ministro

Entonces el conjunto de estados S del ambiente contiene cuatro elementos:

$$S = \{s_1, s_2, s_3, s_4\} = \{\{\sim x, \sim y\}, \{\sim x, y\}, \{x, \sim y\}, \{x, y\}\}$$

Como la habitación es sensitiva a la temperatura, los estados *y* y *~y* son literalmente indistinguibles al termostato.

La función *ver* tendría dos percepciones para el termostato,

ver(s) = - percepción 1 si *s=1* o *s=2* para frío

- percepción 2 si s=3 o s=4 para ok

Agentes con estado

La función de percepción *ver* no cambia y es como sigue:

$ver : S \rightarrow P$

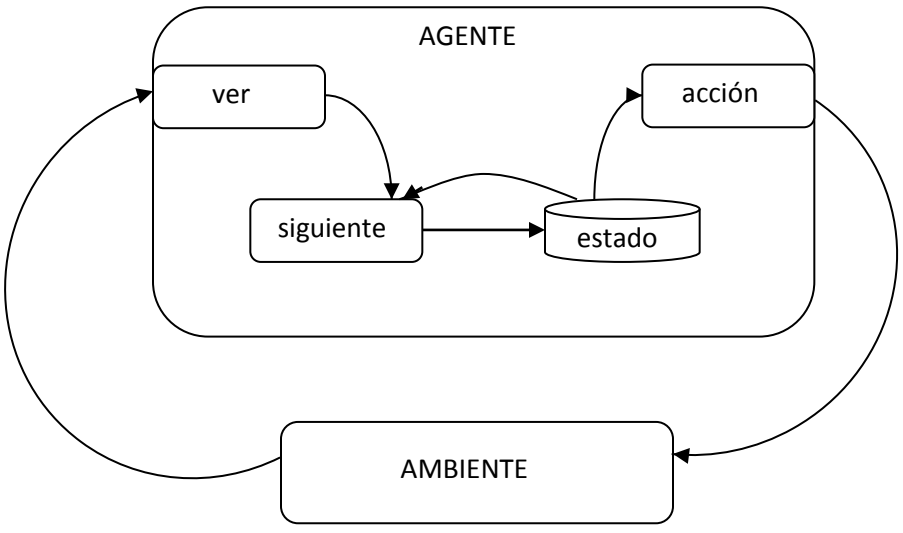
y la función de selección-acción *acción* define ahora un mapeo de estados internos a acciones

$acción : I \rightarrow A$

y se introduce aquí una función adicional *siguiente*, la cual mapea un estado interno y percepción a un estado interno:

$siguiente : I \times P \rightarrow I$

El agente empieza en algún estado interno inicial i_0. Luego observa el estado de su ambiente s, y genera una percepción *ver(s)*. Se actualiza el estado interno del agente vía la función *siguiente*, volviendo el conjunto a *siguiente*(i_0, *ver(s)*). La acción seleccionada por el agente es entonces *acción (siguiente*(i_0, *ver(s)))*. Se realiza la acción y el agente entra a otro ciclo, percibiendo el mundo vía *ver*, actualizando su estado vía *siguiente*, y eligiendo una acción para lleva a cabo vía *acción*.

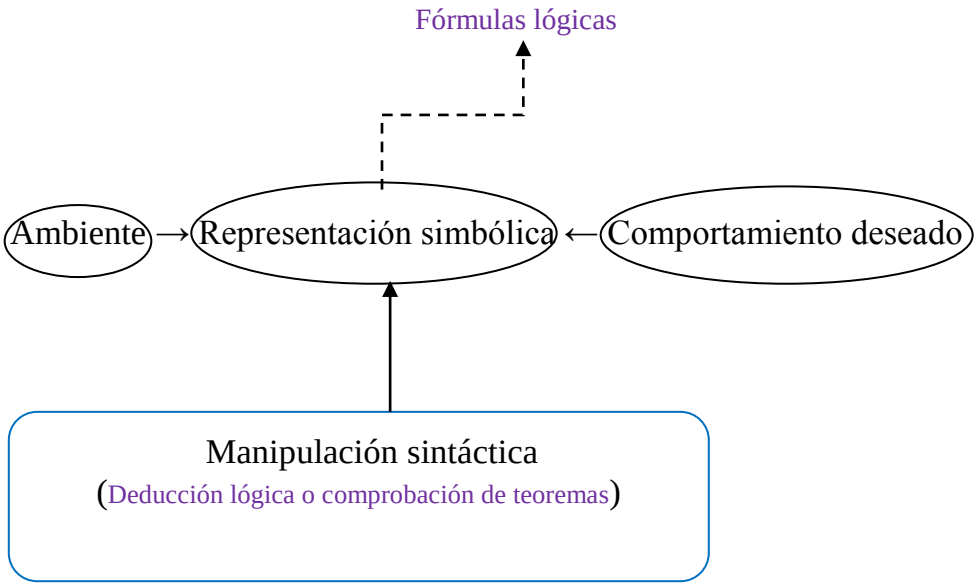


Arquitecturas concretas de los Agentes Inteligentes

Se consideran cuatro clases:

- Agentes basados en lógica
- Agentes reactivos
- Agentes de intensión-deseo-creencia
- Agentes con arquitectura en capas

En arquitecturas basadas en lógica (IA Simbólica)



La base de datos del agente puede contener fórmulas lógicas tales como:

Abrir(*válvula*221)

Temperatura(*reactor*4726, 321)

Presión(*tanque*776, 28)

.....

(conjunto de reglas de deducción) *reglas de inferencia*

~~Es información que el agente tiene de su ambiente y juega un papel similar a la creencia en los humanos. Y es necesario saber la interpretación que el diseñador le da a cada predicado. Se podría escribir:~~

$\Delta \vdash_{\rho} \varphi$ Δ = ~~miembro de la base de datos~~

~~ρ = reglas~~ ~~φ = fórmula por probar~~

La representación simbólica se usa para representar las propiedades de un ambiente.

La base de datos es la información que el agente tiene acerca del ambiente ~ a la creencia de los humanos. Entonces el agente puede tener en su base de datos el predicado

Abierta(válvula221)

Pero no implica que en realidad lo esté porque:

-Sensores con falla

- Razonamiento con falla

- Información no actualizada

- La interpretación propuesta por el diseñador puede ser diferente.

Sea L el conjunto de sentencias de lógica clásica y

D = φ(L) el conjunto de bases de datos (el conjunto de conjuntos de ~~bases de datos~~) y

$\Delta, \Delta_1, \ldots,$ para indicar miembros de D

Un proceso de toma de decisiones de un agente es entonces modelado a través de un conjunto de reglas de deducción ρ (reglas de inferencia para la lógica).

Se describe

$\Delta \vdash_{\rho} \varphi$ Si la fórmula φ Puede ser probada desde la base de datos Δ usando solamente las reglas de deducción ρ.

La función de percepción de los agentes *ver* permanece sin cambios:

ver : S → P y la función siguiente es

siguiente : D x P → D

indica que se mapea una base de datos y una percepción a una nueva base de datos. Y la selección de acción tiene la forma

acción : D → A

Seudocódigo:

```
1.  function action(Δ : D) : A
2.  begin
3.      for each a ϵ A do
4.          if Δ ⊢ρ Do(a) then
5.              return a
6.          end-if
7.      end for
8.      for each a ϵ A do
9.          if Δ ⊬ρ ¬Do(a) then
10.             return a
11.         end-if
12.     end for
```

El agente toma cada una de sus posibles acciones *a* e intenta probar Do(*a*) desde su base de datos, entonces *a* se regresa como la acción por realizar (líneas 1-7)

Cuando falla en todas las acciones $a \epsilon A$. Aquí trata de encontrar una acción que sea consistente o sea que no esté específicamente prohibida. O sea trata de encontrar una $a \epsilon A$ tal que ¬Do(*a*) no pueda ser derivada desde una base de datos usando sus reglas de deducción. Si la encuentra la regresa como acción por realizar, si no regresa *null*

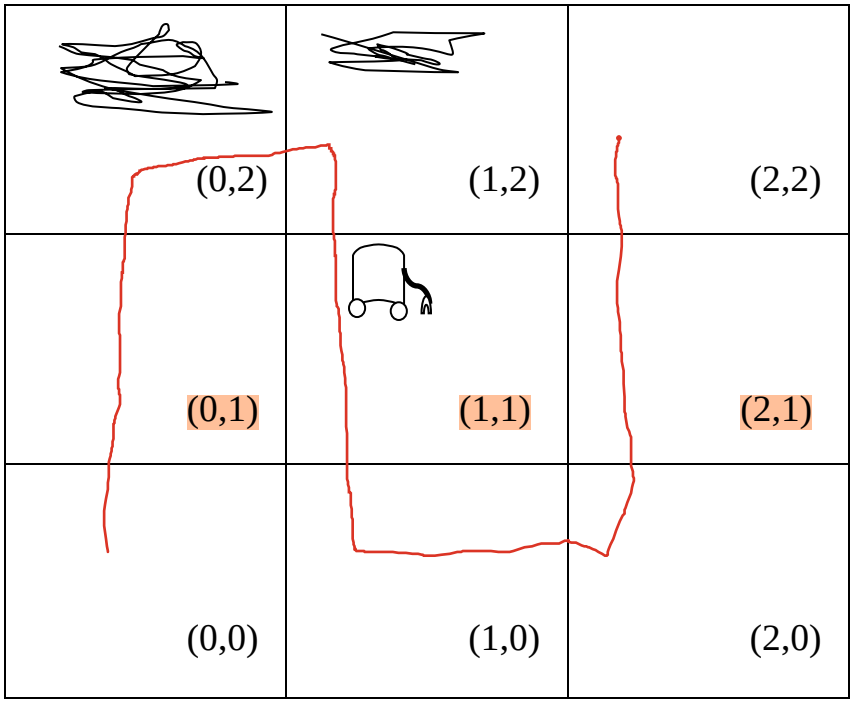
13. return *null*

14. end function action

El comportamiento de un agente está determinado por las reglas de deducción (su programa) y su base de datos actual (la información que el agente tiene de su ambiente).

Ejemplo: Un robot que aspira equipado con:

- Un sensor para saber si está sobre basura
- Un aspirador para succionar
- Siempre tiene una orientación definida (norte, sur, este, oeste)
- Avanza hacia adelante paso a paso
- Gira-derecha 90º
- Rejilla (cuarto cuadriculado)
- No hace otra cosa que limpiar
- Nunca deja el cuarto (de 3x3)
- Inicia siempre en (0,0) cara al norte
- Puede recibir percepción de basura debajo de él, o null indicando que no hay información especial.
- Tres posibles acciones: avanzar, succionar, girar
- Objetivo: atravesar el cuarto continuamente en busca de remover basura



Tres posibles predicados del dominio:

en(x, y)	agente está en (x, y)
basura(x, y)	basura en (x, y)
orientación(d)	agente orientado en dirección d

Especificamos la función *siguiente*. Esta función debe mirar en la información perceptual (*basura* o *null*) obtenida desde su ambiente, y genera una nueva base de datos la cual incluye esta información. En adición borra información vieja o irrelevante. Se especifica en varias partes:

1. Se escribe *vieja*(Δ) para denotar el conjunto de información vieja en una base de datos, la cual queremos que la función de actualización *siguiente* remueva.

   vieja(Δ) = {P(t_1, …, t_n) | P ϵ {en, basura, orientación} y P(t_1, …, t_n) ϵ Δ}

Se lee:
"El conjunto de información *vieja* en una base de datos **delta** es igual a las percepciones *p(t_1, …, t_n)* tal que *p* que pertenece al conjunto *{en, basura, orientación}* y *p(t_1, …, t_n)* que pertenece a **delta**, son verdad.

2. Requerimos una información *nueva*, la cual da el conjunto de predicados nuevos por añadir a la base de datos. Esta función tiene la forma:

   nueva: D x P → D

   Esta genera los predicados *en*(…) que describe la nueva posición del agente, *orientación*(…) que describe la orientación del agente, y *basura*(…) si ha detectado basura en la nueva posición.

Dadas las funciones *nueva* y *vieja*, la función *siguiente* se define como sigue:

siguiente(Δ, p) = (Δ \ vieja(Δ)) ∪ nueva(Δ, p).

Las reglas de deducción tienen la forma: φ(…) → Ψ(…)

φ y Ψ son predicados de una lista de constantes y variables.

Si φ se acopla frente a la base de datos del agente, entonces Ψ puede concluirse, con cualquiera de las variables en Ψ llamadas a ejecución.

Se puede dar prioridad a las reglas, por ejemplo aquí la primera tendría prioridad:

en(x, y) ^ basura(x, y) → Do(aspira) (1.1)

de otra forma el agente viaja por el mundo.

Aprovechando la simplicidad del ambiente, el algoritmo de navegación hará que el robot se mueva desde (0,0) a (0,1) a (0,2) y luego a (1,2), a (1,1) y así …. Cuando llegue a (2,2), debe volver a (0,0).

Entonces las reglas hasta (0,2) son:

$$en(x, y) \wedge basura(x, y) \rightarrow Do(aspira) \quad (1,1)$$

$$en(0,0) \wedge orientación(norte) \wedge \neg basura(0,0) \rightarrow Do(avanzar) \quad (1.2)$$

$$en(0,1) \wedge orientación(norte) \wedge \neg basura(0,1) \rightarrow Do(avanzar) \quad (1.3)$$

$$en(0,2) \wedge orientación(norte) \wedge \neg basura(0,2) \rightarrow Do(girar) \quad (1.4)$$

$$en(0,2) \wedge orientación(este) \rightarrow Do(avanzar) \quad (1.5)$$

Cada regla debe verificar si explícitamente el antecedente de la regla (1,1) se dispara.

Tarea3

Escribir las reglas complementarias que hacen que el agente del robot que aspira (de la figura anterior) llegue a (2,2) y vuelva a (0,0) generando el comportamiento requerido para este caso en función de las características dadas también anteriormente.

Arquitecturas de agentes reactivos

El comportamiento emerge a raíz de la interacción del agente con el ambiente con comportamientos más simples.

Reactivo: reacciona a un ambiente sin razonar acerca de el.

Arquitectura Subsunción.

Según Brooks son máquinas de estados finitos

1. No incluyen representaciones simbólicas complejas

   No incluyen razonamiento simbólico

   Reglas simples *situación ---> acción* (mapeo perceptual directamente a acciones).

2. Muchos comportamientos pueden dispararse simultáneamente

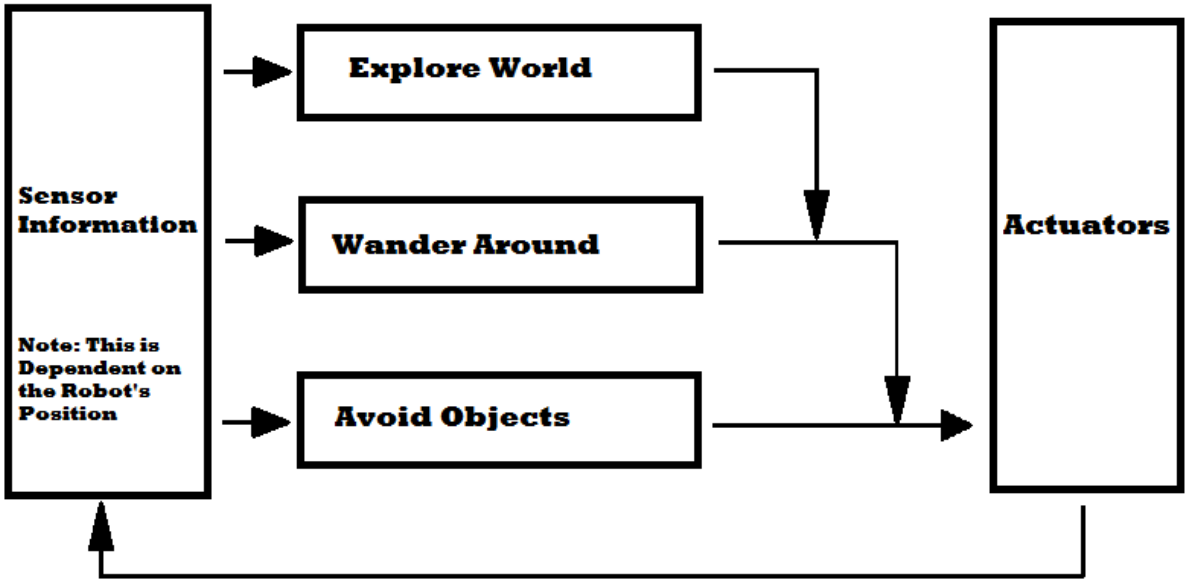


Figure 1: Abstract representation of subsumption architecture, with the higher level layers subsuming the roles of lower level layers when the sensory information determines it.[9]

Un comportamiento es un par (c, a) $c \subseteq P$ es un conjunto de percepciones llamadas la condición.

$a \in A$ es una acción.

Un comportamiento (c, a) se disparará cuando el ambiente esté en el estado $s \in S$ iff

$see \in c$

Sea Beh = $\{(c, a) | \ c \subseteq P$ y $a \in A\}$ el conjunto de tales reglas.

Asociado con un conjunto del agente de reglas de comportamiento $R \subseteq Beh$ es una *relación de inhibición* binaria sobre el conjunto de comportamientos: $\prec \subseteq RXR$.

De este modo $b_1 \prec b_2$ (b_1 inhibe b_2)

Función acción.

1. function *action (p : P) : A*
2. var *fired : P(R)* Conjunto de todas las que se disparan por (5)
3. var *selected : A*
4. begin
5. $fired := \{(c,a) \mid (c,a) \in R \wedge p \in c\}$
6. for each *(c, a)* $\in$ *fired* do
7. *if* $\neg(\exists(c',a') \in$ *fired* such that *(c', a')* $\prec$ *(c, a)* then
8. return *a*
9. end-if
10. end-for
11. return *null*
12. end function *action*

El ejemplo de vehículos autónomos

Varios robots son soltados en un planeta para recoger muestras de rocas y traerlas a la nave nodriza por un tiempo finito. No tienen sistema de comunicación entre ellos, pero la base emite una señal de radio que ellos pueden detectar fuerte o débil de acuerdo a la distancia.

Caso agentes individuales (no cooperan):

El comportamiento de más bajo nivel tiene más alta prioridad (evitar obstáculos)

Comportamiento 1

si detecta un obstáculo *entonces* cambia de dirección (1.6)

Comportamiento 2

si hay muestras transportadas *y* se está en la base *entonces* deja las muestras (1.7)

si hay muestras transportadas *y* no se está en la base *entonces* dirígete a mayor gradiente, hasta el origen. (1.8)

Comportamiento 3

si detecta una muestra *entonces* recógela (1.9)

Comportamiento 4 Si no tiene nada que hacer entonces con la precondición (en la regla siguiente) se supone el siempre dispararse.

si ***true*** *entonces* ***move*** en forma aleatoria (1.10)

La jerarquía sería: $(1.6) \prec (1.7) \prec (1.8) \prec (1.9) \prec (1.10)$

Si el caso es agentes que cooperan entonces el comportamiento 2 se modifica a:

Comportamiento 5

si hay muestras transportadas *y* se está en la base *entonces* deja las muestras (1.11)

si hay muestras transportadas *y* no se está en la base *entonces* (1.12)

deja dos migajas radioactivas y viaja gradiente arriba

Un comportamiento adicional seria:

Comportamiento 6

Si se detectan migajas *entonces* recoje una y viaja gradiente abajo (1.13)

Entonces la jerarquía de subsunción sería:

$(1.6) \prec (1.11) \prec (1.12) \prec (1.9) \prec (1.13 \prec (1.10)$

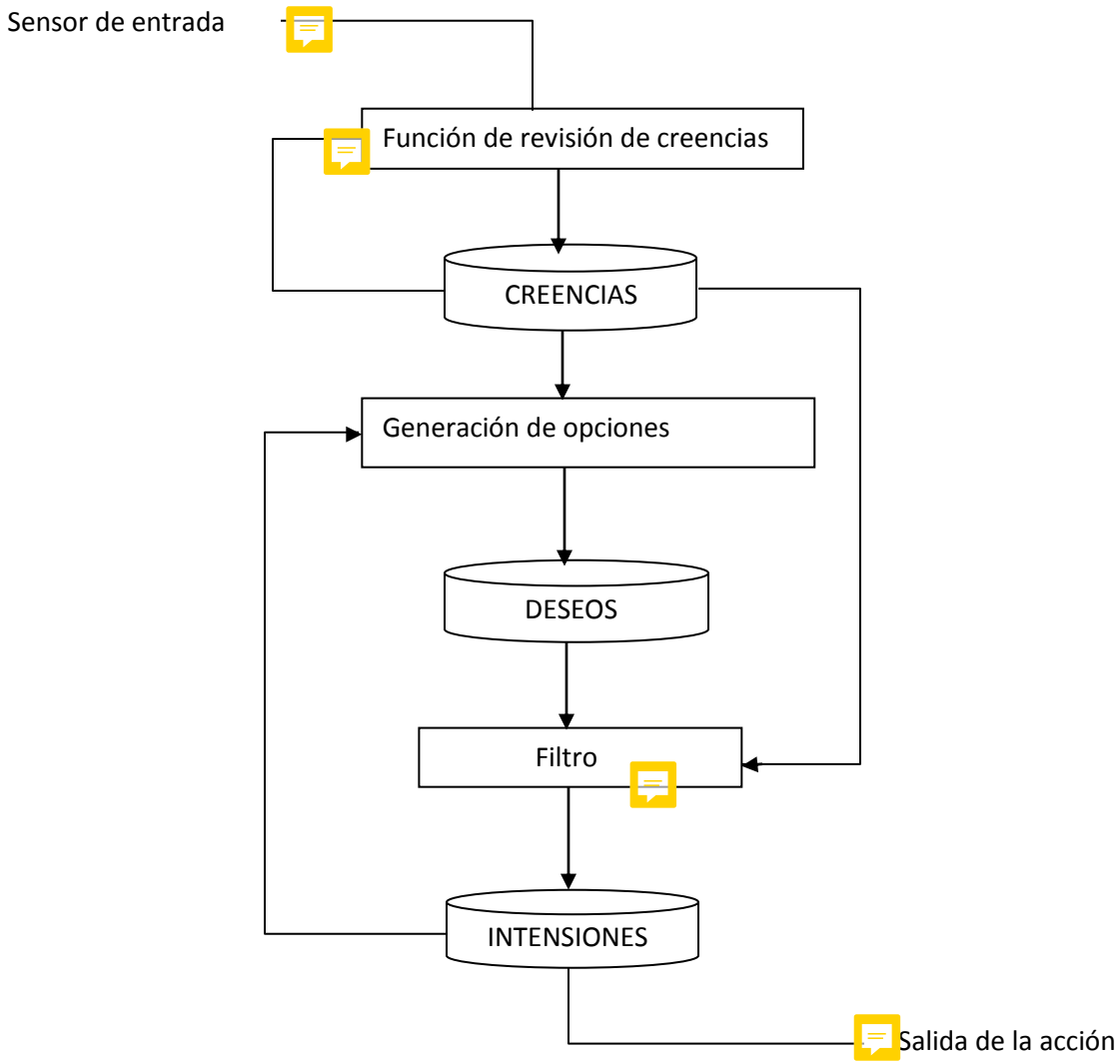
Ventajas: Simplicidad, economía, tratabilidad computacional, robustez, elegancia

Desventajas:

- Si no tienen modelos de su ambiente, entonces deben tener suficiente información local y así determinar una acción recomendable. Inherentemente deben tomar una vista a corta plazo, para tomar decisiones.

- Es difícil ver como agentes puramente reactivos puedan mejorar con la “experiencia”.
- La relación de “emerge” entre lo individual y cooperativo no está bien entendida para poder decir que el comportamiento emerge de la interacción con el ambiente.
- Se dificulta el construir agentes efectivos con capas mayores de 10 para las interacciones necesarias.

Arquitecturas basadas en Intensiones, Deseos, y Creencias (BDI)



Descripción:

La función de revisión de creencias toma como una entrada una percepción y las creencias actuales (conjunto de creencias que el agente tiene acerca del ambiente) y sobre la base de eso, determina nuevas creencias. El generador de opciones determina las opciones disponibles para el agente, con base en sus creencias actuales acerca del ambiente y de sus intenciones actuales. Los deseos son un conjunto de opciones actuales que representan posibles cursos de acción disponibles para el agente. El filtro representa el proceso de deliberación del agente, el cual determina o define las intensiones del agente y con base en ellas se ejecuta una acción.

Formalmente:

Un agente BDI en cualquier momento es:

Una tripleta (B, D, I) donde $B \subseteq Bel$, $D \subseteq Des$, e $I \subseteq Int$

La función de revisión de creencias es:

$$brf : \mathcal{P}(Bel) \times P \rightarrow \mathcal{P}(Bel)$$

donde P = Percepción actual y $\mathcal{P}(Bel)$ = Conjunto de creencias actuales.

La función de generación de opciones es:

$$options : \mathcal{P}(Bel) \times \mathcal{P}(Int) \rightarrow \mathcal{P}(Des)$$ $\mathcal{P}$= Conjunto de

En el filtro:

$$filter : \mathcal{P}(Bel) \times \mathcal{P}(Des) \times \mathcal{P}(Int) \rightarrow \mathcal{P}(Int)$$

Que debe satisfacer:

$$\forall B \in \mathcal{P}(Bel),\ \forall D \in \mathcal{P}(Des),\ \forall I \in \mathcal{P}(Int),\ filter(B, D, I) \subseteq I \cup D$$

La función ejecuta:

$$execute: \mathcal{P}(Int) \rightarrow A$$

La función de decisión acción: $action : P \rightarrow A$

Seudocódigo de esa decisión

1. **function** *action (p : P) : A*
2. **begin**
3. *B := brf(B, p)*
4. *D:= options(D, I)*
5. *I := filter(B, D, I)*
6. **return** *execute(I)*
7. **end function** *action*

Para otras arquitecturas y más sobre agentes ver:

Referencia: Multiagent Systems. *G Weiss*. Ed. Weiss 2000